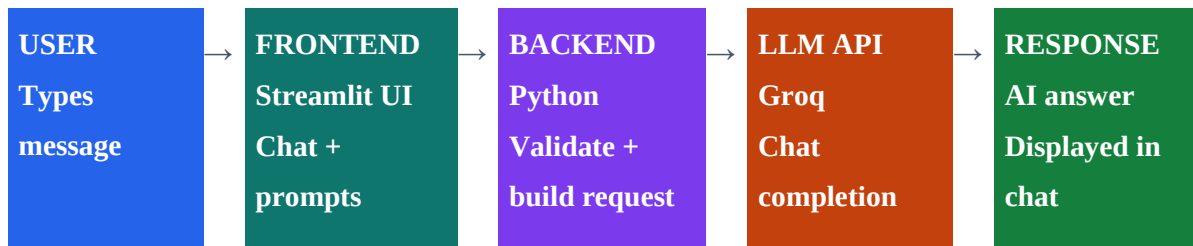


AI APPLICATION ARCHITECTURE

Application: AI Workspace **Frontend:** Streamlit **Backend:** Python **LLM API:** Groq

Architecture Diagram



Request direction: User → Frontend → Backend → Groq LLM API → Response

The backend sends the request to the API and returns the generated answer to the frontend.

1. Request Flow

The user enters a message in the Streamlit frontend. The frontend also provides the selected model, system prompt, and previous chat messages. When the user clicks Send, the Python backend validates the input, combines the conversation history with the system prompt, and creates a chat completion request. The backend sends the request to Groq using an HTTP POST request.

The request includes:

- Selected Groq model
- System prompt, if provided
- Previous conversation messages and new user message
- Maximum response tokens and temperature setting

2. Response Flow

Groq processes the request with the selected language model and returns a JSON response. The Python backend extracts the assistant message from `response["choices"][0]["message"]["content"]`, adds it to the current chat session, and Streamlit displays it in the chat interface. The conversation can also be exported as a text file.

3. Error Handling

The application converts technical failures into understandable messages instead of crashing:

- Empty message: asks the user to enter a message.
- Missing or invalid API key: asks the user to enter a valid Groq key.
- Unavailable model: explains that the model is not accessible to the key.
- Rate limit: tells the user to wait and try again.
- Network or timeout error: displays a connection error.
- Invalid API response: reports that the returned format was unexpected.

4. API Integration

The app uses Groq's OpenAI-compatible API. It loads available models with GET /openai/v1/models and generates responses with POST /openai/v1/chat/completions. The API key is sent in the Authorization: Bearer header. The key should be stored privately as an environment variable or entered in the app; it must never be uploaded to GitHub.

Example request data: { "model": "selected-model", "messages": [...], "max_tokens": 1024, "temperature": 0.7 }

Conclusion

The AI Workspace follows a client-to-API architecture. The user sends a message through Streamlit, Python prepares and sends the request to Groq, and the response returns through Python to the chat interface. Error handling protects the application from invalid input, authentication problems, unavailable models, rate limits, and network failures.